بررسی و ارائه‌ی پیشنهادات در مورد استخراج توابع مجزا با رویکرود برش‌بندی

محمد رحیمی[1]، مسعود رحیمی[2]
[1]دانشگاه علم و صنعت ایران، تهران
rahimi_m72@noor.iust.ac.ir
[2]مرکز آمار ایران، تهران
m.rahimi@shbu.ac.ir

چکیده

یکی از مشکلات طراحی نرم‌افزار توابع چند وظیفه‌ای است. چند وظیفه‌ای بودن یک تابع منجر به افزایش خطوط یک تابع می‌شود که این موضوع مشکلاتی از قبیل درک، خوانایی و آزمون‌پذیری کد را به همراه دارد. یکی از معروف‌ترین رویکردهای استخراج توابع مجزا با عملکرد مستقل، رویکرد مبتنی بر برش است. برش‌های مورد استفاده برای استخراج توابع معمولا برش‌های ایستای روبه‌عقب یا زیرمجموعه‌ای از برش‌های ایستای روبه عقب هستند. دو سوال در اینجا مطرح می‌شود: ۱- آیا می‌توان از برش‌های روبه‌جلو برای استخراج توابع استفاده کرد؟ ۲- آیا امکان جایگزینی برش‌های دیگر به جای برش ایستا وجود دارد؟ که در این مقاله مفاهیم و تعدادی از پرکاربردترین برش‌ها معرفی شده است و به سوالات ذکر شده پاسخ داده می‌شود.

کلمات کلیدی

استخراج تابع، برش‌بندی، برش ایستا، برش پویا، برش روبه‌عقب

۱- مقدمه

نرم‌افزارها پس از طراحی، لازم است مورد آزمون قرار گیرند. به تلاش‌هایی که در جهت عیب‌یابی و رفع مشکلات نرم‌افزار انجام می‌شود آزمون نرم‌افزار می‌گویند. برای آنکه یک نرم‌افزار بتواند مورد آزمون قرار بگیرد پیشنهاد می‌شود مفاهیم و نکات مربوط به کدنویسی تمیز در آن رعایت شود. کدنویسی تمیز مجموعه‌ای از اصول و قوانین است که فهم و توسعه‌ی کد را توسط توسعه‌دهنده ساده می‌کند. به طور کلی خوانایی کد با رعایت این نکات بهبود می‌یابد. ایده‌ی کدنویسی تمیز توسط آقای رابرت سی مارتین مطرح شد. ایشان اصول نوشتن کدنویسی تمیز را در کتاب «کلین کد[6] » مطرح نمودند. از جمله اصول مطرح شده که مورد توجه برنامه‌نویسان قرار گرفت اصول پنج‌گانه‌ی SOLID است. اولین اصل سالید که به اختصار SRP(Single Responsibility Principle) نامیده می‌شود. این اصل بیان می‌کند که یک کلاس یا تابع باید یک وظیفه داشته باشد که در این صورت این کلاس یا تابع فقط به خاطر تغییر در وظیفه دستخوش تغییر خواهد شد[6]. بنابراین یکی از معیارهای تست‌پذیری کد این است که توابع و کلاس‌های کد تک وظیفه‌ای باشند. حال سوالی که در اینجا مطرح می‌شود این است که چه معیاری می‌تواند به عنوان یک واحد عملیاتی، درون کد شناخته شود؟ به این معنی‌که یک واحد عملیاتی از چه نقطه‌ای شروع و در چه نقطه‌ای پایان می‌پذیرد؟ در واقع این یکی از نقدهایی است که توسط آقای دیوید برایان کوپلند در کتاب «Solid is not solid [20]» به اصل اول سالید وارد کرده است. در پاسخ می‌توان گفت برش(Slice)

قابل اجرا با مجموعه‌ای از شرایط خاص می‌تواند به عنوان یک واحد عملیاتی در نظر گرفته شود. یکی از این شرایط آن است که رفتار تابع چند وظیفه‌ای پس از بازسازی حفظ شود. حال در این مقاله به بررسی نحوه‌ی استخراج توابع مستقل با استفاده از برش‌بندی پرداخته می‌شود و چند سوال مهم که در ذهن نویسندگان مقاله ایجاد شده است پاسخ داده می‌شود. این سوالات عبارتند از:

۱- چرا برای استخراج تابع تک وظیفه‌ای از برش ایستا در مقالات استفاده می‌شود[8,21] در حالیکه معیارهای ارزیابی برش‌بندی، برش پویا(Dynamic slice) یا مشتقاتی از این نوع برش را برای این‌کار پیشنهاد می‌کنند؟

۲- چرا در استخراج توابع مجزا از یک برش ایستای روبه‌عقب استفاده می‌شود؟[8,21] در حالیکه یک برش ایستا می‌تواند در جهت روبه‌جلو نیز محاسبه شود.

۳- رویکردهای مبتنی بر برش چه مزایایی نسبت به سایر رویکردهای استخراج متد دارند؟

برای پاسخ به این سوالات لازم است مفاهیم و کاربردهای برش‌بندی معرفی شوند.

برنامه‌ی برش‌بندی شده همه‌ی عباراتی را شامل می‌شود که بر مقدار متغیر در برخی از نقاط برنامه تاثیر گذار است. البته تعریف برش در مقالات مختلف با یکدیگر اندکی تفاوت دارد. دلیل این تفاوت می‌تواند به شاخص برش(Slicing Criterion) و انواع آن برگردد. شاخص برش موقعیتی را نشان می‌دهد که برش از آن نقطه محاسبه می‌شود به عبارت دیگر این شاخص به شماره‌ی خط و نام متغیر برای عملیات برش نیاز دارد.

برش‌بندی کد در درک برنامه، اعتبارسنجی برنامه، محاسبه‌ی اتوماتیک چند معیار مهندسی نرم‌افزار، نگهداری و آزمایش نرم‌افزار، انسجام کد، حذف کدهای مرده(بدون استفاده)، موازی‌سازی برنامه‌های متوالی، قابلیت حمل نرم‌افزار، تولید مولفه با قابلیت استفاده‌ی مجدد، بهینه‌سازی کامپایلر، یکپارچه‌سازی برنامه، تضمین کیفیت نرم‌افزار، عیب‌یابی نرم‌افزار، استخراج توابع تک وظیفه‌ای از توابع چند وظیفه‌ای و ... مورد استفاده قرار می‌گیرد[3,7,21,23,25,29].

به عنوان مثال جدول ۱ را در نظر بگیرید. این جدول تابع statement را نشان می‌دهد. این تابع میزان دریافت اجاره‌بها را از مستاجرانی محاسبه می‌کند که یک فیلم را از قبل اجاره کرده‌اند. جدول 2 یک برش ایستا(Static slice) از این کد را بر اساس متغیر frequentRenterPoints نشان می‌دهد. یک برش می‌تواند ایستا، پویا، قابل اجرا، غیرقابل اجرا، روبه‌جلو(Forward) و روبه‌عقب(Backward) باشد. هنگام برش‌بندی چنانچه از اطلاعات ایستا استفاده شود آن برش ایستا نامیده می‌شود و اگر از اطلاعات پویا استفاده شود آن برش پویا نامیده می‌شود[3] . یک برش می‌تواند ایستا یا پویا یا ترکیبی از هر دو ویژگی است.

برش ایستای یک متغیر خاص، همه‌ی محاسبات و مسیرهای اجرای آن متغیر را در برنامه پوشش می‌دهد در حالیکه در یک برش پویا با توجه به مقدار ورودی متغیر، مسیر خاصی از محاسبات مربوط به آن متغیر در نظر گرفته می‌شود.

Number	Code
1	public String statement ()
	{
2	double totalAmount = 0;
3	int frequentRenterPoints = 0;
4	Enumeration rentals = _rentals. elements ();
5	String result = "Rental Record\n";
6	while (rentals. hasMoreElements ())
	{
7	Rental each= (Rental)rentals.nextElement();
8	double thisAmount = each. get Charge ();
9	if (each.getDaysrented>1)
	frequentRenterPoints =
10	frequentRenterPoints + 2;
	else
11	frequentRenterPoints ++;
12	result = result + each. GetMovie().getTitle() + "\t" + String.valueOf(thisAmount) + "\n";
13	totalAmount = totalAmount + thisAmount;
	}
14	result += "Amount owed id: " + String.valueOf(totalAmount) + "\n";
15	result = "you earned "+ String.valueOf(frequentRenterPoints) + "frequent renter points";
16	return result;
	}

جدول ۱: تابع statement

ویژگی دیگری که مورد بررسی قرار می‌گیرد جهت برش است. جهت می‌تواند روبه‌عقب یا جلو باشد. اصطلاح برش روبه‌جلو اولین بار توسط رپس و بیکر مطرح شد[22]. برش روبه‌جلو حاوی عباراتی از برنامه است که تحت تاثیر شاخص برش قرار می‌گیرند در حالیکه برش روبه‌عقب حاوی عباراتی از برنامه است که در معیار برش تاثیر گذار است[2]. این دو ویژگی در برابر یکدیگر قرار دارند یعنی آنکه برش می‌تواند روبه‌جلو یا روبه‌عقب باشد.

قابل اجرا یا غیر قابل اجرا بودن، ویژگی دیگری است که مورد بررسی قرار می‌گیرد. برش قابل اجرا، کد تولید شده بعد از عملیات برش‌بندی است که بعنوان برنامه کامپایل و اجرا می‌شود[18] . برش‌های قابل اجرا نیز در برابر برش‌های غیرقابل اجرا قرار دارند.

برش‌ها انواع متفاوتی دارند در این مقاله پرکاربردترین و مهمتری برش‌ها مورد بررسی قرار گرفته است. برش‌های مورد بررسی عبارتند از: برش ایستا، برش پویا، برش ایستای کواسی(Quasi static slicing)، برش پویای همزمان، برش آمورفوس(Amorphous slicing)، برش‌بندی مبتنی بر بلوک.

برش ایستا اولین بار توسط وایزر و همکارانش معرفی شد[1]. برش ایستا عباراتی از برنامه را شامل می‌شود که بر مقدار متغیر در نقطه‌ی مورد نظر تاثیرگذار هستند.

پس از آن کرل و همکارانش [19] تحقیقاتی را در خصوص برش‌های پویا انجام دادند. آن‌ها عقیده دارند با توجه به اینکه در فرآیند تست و اشکال‌زدایی به طور خاص برنامه با اجراهای نادرستی روبه‌رو می‌شود معمولا توسعه‌دهندگان نرم‌افزار علاقه‌مند به یافتن علت خطای اجرا هستند که به این دلیل لازم است برش، رفتار برنامه را حفظ کند نه آنکه مجموعه‌ای از ورودی‌ها وجود داشته باشد که اجرای برنامه به خاطر

آن‌ها با شکست روبرو شود. در این مقاله نویسنده معتقد است برش ایستا حاوی عباراتی است که تاثیری بر مقدار متغیر مورد نظر ندارد در حالی‌که با در نظر گرفتن آنالیز پویا می‌توان عباراتی را که بر مقدار متغیر تاثیرگذار نیستند به صورت ساده‌تر مشخص کرد.

برش مورد بررسی دیگر، برش ایستای کواسی است. این برش اولین بار توسط ونکاتش معرفی شد[13] . برش کواسی ترکیبی از برش‌های پویا و ایستا بوده و از ویژگی‌های هر دو برش در ساخت آن استفاده می‌شود. در برش کواسی مقدار برخی متغیرها ثابت و مقدار برخی قابل تغییر است که منجر به تحلیل برنامه می‌شود [27].

هال رویکرد متفاوتی را با توجه به مجموعه اجراهای برنامه برای برش‌بندی تعریف کرد[5] . این تکنیک برش‌بندی از ترکیب موارد آزمون(Test case) به همراه برش‌های برنامه ایجاد می‌شود. با توجه به آنکه این تکنیک، برش‌های پویا را گسترش می‌دهد و به طور همزمان اعمال می‌شود برش پویایی همزمان نامیده می‌شود.

برش مورد بررسی دیگر آمورفیس است که توسط هارمن و همکارانش ارائه شد. برش‌های ایستا، پویا، پویای همزمان و کواسی برش‌هایی با محدودیت نحوی هستند در حالیکه برش آمورفیس بر مبنای محدودیت‌های معنایی برنامه ساخته شده است[15 ,14].

برش مبتنی بر بلوک برش دیگری است که اولین بار توسط ماریاما ارائه شد[8] . برش مبتنی بر بلوک، زیرمجموعه‌ای از برش ایستاست که با ساخت مناطق مبتنی بر بلوک، برش ایستا را در مرزهای یک بخش محدود می‌کند. این رویکرد با رعایت نکات و شرایط خاص، کوچکترین برش را برای متغیر انتخاب شده ارائه می‌کند و همانطور که وایزر و همکارانش [1] اشاره کردند هر چه برش کوچکتر باشد آن برش بهتر است.

پس از معرفی مفاهیم مربوط به برش‌بندی و انواع برش نوبت به محاسبه‌ی برش مبتنی بر بلوک می‌رسد. همانطور که گفته شد یک برش مبتنی بر بلوک زیر مجموعه‌ای از برش‌های ایستا است و بخش عمده‌ای از محاسبات این دو نوع از برش با یکدیگر همپوشانی دارند.

به طور کلی برای محاسبه‌ی برش مبتنی بر بلوک لازم است گراف وابستگی برنامه که به اختصار PDG(Program Dependency graph) نوشته، محاسبه می‌شود. گراف وابستگی برنامه اولین بار توسط فرانت و همکارانش معرفی شد[29]. در این گراف، هر راس یک عبارت برنامه و یال‌های گراف، وابستگی‌های داده‌ای و کنترلی را مشخص می‌کنند. برای تعیین وابستگی‌های داده‌ای لازم است زنجیره‌های تعریف و استفاده‌ی متغیرها درون تابع محاسبه شود. چنانچه مقدار متغیر به وسیله‌ی انتساب اصلاح یا مقداردهی مجدد شود این متغیر تعریف شده است اما چنانچه مقدار متغیر خوانده شود در اصطلاح آن متغیر استفاده شده است. لازم است مجموعه متغیرهای تعریف و استفاده شده درون هر عبارت مشخص شوند. یک لبه‌ی وابستگی داده‌ای از گره‌ی p به گره‌ی q ناشی از متغیر x نشان می‌دهد که عبارت p متغیر x را تعریف کرده و عبارت q از متغیر x استفاده می‌کند به طوریکه مسیر جریان کنترلی از عبارت p به q وجود داشته باشد بدون آنکه x در این مسیر مجددا تعریف شود. مفهوم دیگری که مورد بررسی قرار می‌گیرد لبه‌ی وابستگی کنترلی است. لبه‌ی وابستگی کنترلی از عبارت p به عبارت q نشان می‌دهد که اجرای گره‌ی q به موقعیت کنترلی عبارت p وابسته است.

در گام بعد لازم است برش ایستای روبه‌عقب محاسبه شود. برای محاسبه‌ی برش ایستا کافی است گره‌ی متناظر با عبارت مورد اشاره در شاخص برش مربوط به گراف وابستگی برنامه یافت شود و گراف به سمت عقب پیمایش شود به طوریکه در این پیمایش تمام گره‌هایی که به طور مستقیم یا غیر مستقیم با گره‌ی مورد نظر وابستگی داده‌ای و کنترلی دارند علامت‌دار شوند. در پایان گره‌های علامت‌دار شده عبارات برش ایستا را تشکیل می‌دهند.

پس از محاسبه‌ی برش ایستا کافیست در هر تابع، مناطق مربوط به آن تابع مشخص شود و سپس عبارات برش ایستا محدود به هر منطقه شوند. با این رویکرد برش‌های ایستا در هر منطقه بدست می‌آیند.

۲- رویکردهای استخراج تابع مبتنی بر برش‌بندی

برش‌بندی یکی از معروف‌ترین رویکردهای استخراج تابع تک‌وظیفه‌ای از توابع چند وظیفه‌ای است. نکته‌ی مبهمی که در این موضوع وجود دارد اندازه‌ی تابع است. در واقع اندازه‌ی تابع برابر با تعداد جملات یا تعداد خطوط کد است که برای نوشتن تابع به کار می‌رود. معمولا آستانه‌ی مشخصی برای تعیین بزرگی یک تابع وجود ندارد و این آستانه در مقالات مختلف متفاوت است. به طور مثال در مقاله‌ی نیکولاس تسانتالیس و همکارانش [21] تعداد خطوط یک تابع بزرگ بیشتر از 10 جمله در نظر گرفته شده است. در این بخش انواع برش به طور کامل تشریح و به سوالات زیر پاسخ داده می‌شود.

۱- چرا در استخراج توابع مجزا از یک برش ایستای روبه‌عقب استفاده می‌شود؟[8,21] در حالیکه یک برش ایستا می‌تواند در جهت روبه‌جلو نیز محاسبه شود.

۲- چرا برای استخراج تابع تک وظیفه‌ای از برش ایستا در مقالات استفاده می‌شود[8,21] در حالیکه معیارهای ارزیابی برش‌بندی، برش پویا یا مشتقاتی از این نوع برش را برای این‌کار پیشنهاد می‌کنند؟

۳- رویکردهای مبتنی بر برش چه مزایایی نسبت به سایر رویکردهای استخراج متد دارند؟

برای پاسخ به سوال اول لازم است جهت برش بررسی شود.

۲-۱- بررسی ویژگی جهت برش

همانطور که گفته شد جهت برش روبه‌جلو یا روبه‌عقب است. برش روبه‌جلو حاوی عباراتی از برنامه است که تحت تاثیر شاخص برش قرار می‌گیرند. برای درک این مفهوم لازم است از یک مثال استفاده شود. قطعه کد جدول ۱ را در نظر گرفته و برش ایستای روبه‌جلو برای متغیر frequentRenterPoints محاسبه می‌شود. حاصل برش به ترتیب عبارات ۳، ۶، ۹، ۱۰، ۱۱، ۱۵ است. جدول شماره‌ی ۲ این عبارات را نشان می‌دهد.

Number	Code
3	int frequentRenterPoints = 0;
6	while (rentals. hasMoreElements ())
	{
9	if (each.getDaysrented>1)
	frequentRenterPoints =
10	frequentRenterPoints + 2;
	else
11	frequentRenterPoints ++;
	}
15	result = "you earned "+
	String.valueOf(frequentRenterPoints) +
	"frequent renter
	points";
	}

جدول ۲:برش ایستای روبه‌جلو برای متغیر frequentRenterPoints

همانطور که مشاهده می‌شود برش روبه‌جلو علاوه بر تاثیر بر روی عبارات خطوط ۱۰ و ۱۱ که متغیر frequentRenterPoints در آن خطوط تعریف شده، بر روی عبارت خط شماره‌ی ۱۵ که متغیر result در آن تعریف شده، تاثیر گذاشته است. در نتیجه اگر این عبارات به تابعی مجزا تبدیل شوند هر دو متغیر result و frequentRenterPoints به عنوان خروجی تابع بازگردانده می‌شوند که این کار عملا غیر ممکن است. چرا که هر تابع یک متغیر را به عنوان خروجی برمی‌گرداند.

برش روبه‌جلو با تخمین خطای ناشی از تغییر مقدار متغیر در برنامه و بررسی ایجاد تاثیرات موج‌دار(ripple effects) در تخمین خطا به توسعه‌دهنده کمک می‌کند. به عبارت دیگر برش روبه‌جلو می‌تواند به توسعه‌دهنده کمک کند تا موقعیت خطا را در برنامه پیدا و آن را اصلاح کند. در این موارد توسعه‌دهنده تاثیرات موج‌دار حاصل از تغییر متغیر را در برنامه بررسی می‌کند. برای این منظور اصطلاح تاثیرات موج‌دار تعریف می‌شود. تاثیر موج‌دار در برنامه زمانی اتفاق می‌افتد که چندین متغیر به یک متغیر وابسته هستند و با تغییر مقدار متغیر سایر متغیرها تحت تاثیر قرار می‌گیرند. در این موارد به وسیله‌ی برش‌بندی روبه‌جلو تمامی عباراتی که تحت تاثیر این متغیر قرار می‌گیرند در برنامه پیدا کرده و بعد از تغییر مقدار متغیر، این جملات بررسی می‌شوند و در صورت بروز خطا عملیات اشکال‌زدایی انجام می‌شود [2].

به طور مثال در برش ایستای روبه‌جلوی متغیر frequentRenterPoints ، جملات خطوط ۱۰، ۱۱ و ۱۵ تحت تاثیر قرار می‌گیرند از این رو چنانچه توسعه‌دهنده مقدار متغیر frequentRenterPoints را تغییر دهد امکان بروز خطا در جملات ذکر شده وجود دارد. بنابراین توسعه‌دهنده پس از تغییر متغیر، عبارات ۱۰، ۱۱ و ۱۵ را بررسی می‌کند تا چنانچه خطایی ایجاد شود این خطا را برطرف کند.

اما برش ایستای روبه‌عقب مشکل گفته شده در برش روبه‌جلو را ندارد. برش روبه‌عقب حاوی عباراتی از برنامه است که در معیار برش تاثیر گذار است. برای درک این جمله برش ایستای روبه‌عقب از متغیر frequentRenterPoints محاسبه می‌شود. برش محاسبه شده شامل عبارات ۲، ۴، ۶، ۷، ۹، ۱۰، ۱۱ است. همانطور که مشاهده می‌شود متغیر frequentRenterPoints تنها در عبارات ۱۰ و ۱۱ تاثیر گذار است چنانچه این عبارات به یک تابع مجزا تبدیل شوند تنها مقدار برگشتی توسط این تابع frequentRenterPoints است و مشکل ذکر شده را در برش روبه‌جلو ندارد.

متغیر each در عبارت خط شماره‌ی ۷ را در نظر بگیرد. این متغیر در شرط کنترلی if عبارت خط شماره ۹ استفاده شده است که در برش جدول شماره‌ی ۳ قرار دارد.

Number	Code
3	int frequentRenterPoints = 0;
4	Enumeration rentals = _rentals. elements ();
6	while (rentals. hasMoreElements ())
	{
7	Rental each = (Rental)rentals.
	nextElement ();
9	if (each.getDaysrented>1)
10	frequentRenterPoints =
	frequentRenterPoints
	+ 2;
	else
11	frequentRenterPoints ++;
	}

جدول ۳:برش ایستای روبه‌عقب بر اساس متغیر frequentRenterPoints

۲-۲- برش ایستا

برش ایستا اولین بار توسط وایزر و همکارانش معرفی شد[1]. برش ایستا عباراتی از برنامه را شامل می‌شود که بر مقدار متغیر در نقطه‌ی مورد نظر تاثیرگذار باشد. وایزر و همکارانش عقیده دارند که هر چه برش ایستا کوچکتر باشد آن برش بهتر است اما پیدا کردن برش ایستا با حداقل عبارات مسئله غیر قابل حل است. با این وجود تحلیل جریان داده که به اختصار DFA(Data flow analysis) نامیده می‌شود در پیدا کردن برش‌های تقریبی حداقلی، مناسب است. چنانچه عملیات برش‌بندی بر روی رویه انجام شده و آن رویه مورد فراخوانی رویه‌های دیگر قرار گیرد یا رویه‌هایی را فراخوانی کند احتمالا لازم است برخی از عبارات درون رویه‌های فراخوانی‌شده یا فراخواننده تکرار گردند(حفظ شوند). همچنین فراخوانی رویه‌های خارجی توسط برش یا فراخوانی شدن برش توسط رویه‌های خارجی می‌تواند مشکلاتی در محاسبه‌ی برش ایجاد نماید. با توجه به آنکه برای محاسبه‌ی برش مبتنی بر بلوک ابتدا برش ایستا محاسبه می‌شود در بخش ۸-۲ نحوه‌ی محاسبه‌ی برش ایستا به عنوان بخشی از محاسبات برش مبتنی بر بلوک تشریح می‌شود.

شاخص برش ایستای $C = \langle p , v \rangle$ برش ایستا را با توجه به نقطه‌ی p و متغیر v محاسبه می‌کند به‌طوریکه لازم نیست در این نقطه متغیر p تعریف یا استفاده شود[1] . البته این تعریف با توصیف شاخص ایستا در مقاله‌ی هارویتز متفاوت است[26]. به طوریکه با توجه به توصیف هارویتز لازم است متغیر v در نقطه‌ی p تعریف یا استفاده شود. جدول شماره‌ی ۳ شاخص برش $\langle 11 , \text{frequentRenterPoints} \rangle$ را نشان می‌دهد.

۳-۲- برش پویا

در خصوص برش‌های پویا، کرل و همکارانش [19] عقیده دارند که توسعه‌دهنده یا آزمون‌گر نرم‌افزار، هنگام عملیات تست و اشکال‌زدایی با

اجراهای نادرستی از برنامه مواجه است و معمولاً علاقه‌مند به یافتن علت خطاهای زمان اجرا است. در این مقاله نویسنده معتقد است برش ایستا، حاوی عباراتی است که تاثیری بر مقدار متغیر مورد نظر ندارد در حالی‌که با در نظر گرفتن آنالیز پویا می‌توان عباراتی را که بر مقدار متغیر تاثیرگذار نیستند به صورت ساده‌تری مشخص نمود. همچنین در این مقاله به نحوه اداره کردن آرایه‌ها در برش، به صورت پویا پرداخته شده است. در رویکرد ایستا آرایه‌ی کامل به عنوان یک متغیر در نظر گرفته می‌شود یعنی آنکه تعریف یا استفاده از هر کدام از عناصر آرایه به منزله‌ی تعریف یا استفاده از آرایه‌ی کامل است با وجود آنکه اجرای این روش آسان است اما هرگونه اطلاعاتی را در مورد عناصر خاص آرایه در نظر نمی‌گیرد که منجر به درج عباراتی می‌شود که هیچ تاثیری روی مقادیر خاصی از عناصر آرایه ندارد. در این مقاله هر عنصر آرایه، به عنوان متغیر جداگانه‌ای در نظر گرفته شده است.

شاخص برش پویا، به صورت $C = \langle x,p,v \rangle$ نمایش داده می‌شود که برش پویا را با اجرای برنامه بر روی ورودی x با در نظر گرفتن نقطه‌ی p و متغیر v محاسبه می‌کند به طوریکه متغیر v در این نقطه از برنامه تعریف یا استفاده نمی‌شود [3]. یکی از تفاوت‌های شاخص برش ایستا و پویا آن است که برش پویا مقدار ثابتی را به عنوان ورودی برای متغیری مانند x در نظر می‌گیرد و عملیات برش‌بندی را بر روی متغیری مانند v انجام می‌دهد این در حالی است که برش ایستا هیچ مقدار ثابتی را برای متغیر x در نظر نمی‌گیرد. به طور مثال جدول ۴ برش پویای روبه‌عقب را با توجه به شاخص زیر نشان می‌دهد.

⟨each.getRenterPoints=2,15, frequentRenterPoints⟩

Number	Code
3	int frequentRenterPoints = 0;
4	Enumeration rentals = _rentals. elements ();
6	while (rentals. hasMoreElements ())
	{
7	Rental each = (Rental)rentals. nextElement ();
9	if (each.getDaysrented>1)
10	frequentRenterPoints =
	frequentRenterPoints +
	2;
	}

جدول ۴: برش‌بندی پویای متغیر frequentRenterPoints

همانطور که مشاهده می‌شود این برش در مقایسه با برش ایستای مشابه آن کوچکتر است زیرا عبارت شماره‌ی ۱۱ از این برش حذف شده است. دلیل حذف این عبارت آن است که مقدار ورودی متغیر each.getRenterPoints=2 در نظر گرفته شده است و بنابراین مسیر اجرا از عبارت شماره‌ی ۱۱ جدول ۳ عبور نمی‌کند.

۴-۲- مقایسه برش ایستا و پویا

با توجه به نظر کرل و همکارانش [19] ، برخی از مزایای برش پویا نسبت به برش ایستا عبارتند از:

- اندازه‌ی برش به طور قابل توجهی کاهش می‌یابد به طوریکه هنگام محاسبه این نوع از برش‌بندی، عباراتی که بر روی متغیر مورد نظر تاثیری ندارند از برش حذف می‌شوند.
- در نظر گرفتن متغیرهای اشاره‌گر یکی دیگر از مزایای برش‌بندی پویا نسبت به برش ایستا است چراکه متغیرهای اشاره‌گر دو متغیر را نشان می‌دهند: اشاره‌گر و شی مورد اشاره. شی مورد اشاره با فراخوانی یک رویه استاندارد، ایجاد می‌شود که اعلان پویایی از این شی است. به عبارتی فضای ذخیره‌سازی به این شی اختصاص می‌یابد اما هیچ مقداری به آن انتساب داده نمی‌شود. حال شناسایی این اشیای پویا در برنامه با استفاده از تحلیل ایستا غیر ممکن است. در مقابل با استفاده از تحلیل پویا، لیستی از متغیرهای پویا در هنگام اجرای برنامه ایجاد و دستکاری می‌شود. با این لیست می‌توان مشخص کرد که کدام اشیا در هر مرحله از اجرای برنامه استفاده یا اصلاح می‌شوند. همچنین مشخص می‌شود کدام اشیای پویا به عنوان متغیرهای اشاره‌گر در هر نقطه از برنامه مورد اشاره قرار می‌گیرند.

۵-۲- برش ایستای کواسی

برش ایستای کواسی سومین برشی است که مورد بررسی قرار می‌گیرد. این برش اولین بار توسط ونکاتش معرفی شد[4] و ترکیبی از برش‌های ایستا و پویا است. برش ایستا در طول زمان اجرا بررسی می‌شود. به طوریکه از اطلاعات ورودی متغیرها (مقادیر متغیرها) استفاده نمی‌کند در حالیکه برش‌بندی پویای کد، به وسیله‌ی ورودی‌های داده شده به برنامه، تحلیل می‌شود. برش‌بندی پویا در زمان اجرای برنامه با ورودی‌های داده شده ساخته می‌شود. برش‌بندی ایستا احتیاج به فضا و منابع بیشتری نسبت به برش پویا دارد و هر اجرای ممکن از برنامه را در نظر می‌گیرد در حالیکه برش پویا به منابع و فضای کمتری احتیاج دارد و به طور مخصوص برای اجرای برنامه در نظر گرفته می‌شود. همانطور که گفته شد برش پویا از برش ایستای مشابه خود کوچکتر است[9]. در برش کواسی مقدار برخی متغیرها ثابت و مقدار برخی تغییر می‌کنند و با توجه به این برش برنامه تحلیل می‌شود. رفتار برنامه‌ی اصلی با توجه به شاخص برش تغییر نمی‌کند. شاخص برش مجموعه‌ای از متغیرهای مورد نظر و وضعیت اولیه آن‌هاست و از این رو شاخص کواسی با عنوان شاخص حالت نیز شناخته می‌شود[9]. این رویکرد عالی برای درک برنامه است. هنگامی که اثبات شود برش‌های پویا و ایستا رفتار یکسانی داشته باشند این تکنیک با شکست مواجه می‌شود.

شاخص برش کواسی به صورت ⟨variable1,1,13,variable2⟩ است. اولین آرگومان این شاخص متغیری با مقدار ثابت را نشان می‌دهد. دومین آرگومان شاخص، مقدار متغیر آرگومان اول را نشان می‌دهد. سومین آرگومان شماره‌ی عبارتی را نشان می‌دهد که برش‌بندی از آنجا آغاز می‌شود و چهارمین آرگومان نشان‌دهنده متغیری است که هیچ محدودیتی در هنگام برش‌بندی برای آن وجود ندارد. با در نظر گرفتن کد جدول ۱ و عملیات برش‌بندی با شاخص برش کواسی زیر نتایج جدول ۵ قابل مشاهده است.

⟨each.getRenterPoints,2,14,frequentRenterPoints⟩

Number	Code
3	int frequentRenterPoints = 0;
4	Enumeration rentals = _rentals. elements ();
6	while (rentals. hasMoreElements ())
	{
7	Rental each = (Rental)rentals. nextElement();
9	if (each.getDaysrented>1)
10	frequentRenterPoints = frequentRenterPoints + 2;
	}

جدول ۵: برش کواسی برای متغیر frequentRenterPoints

۶-۲- برش پویای همزمان

هال رویکرد متفاوتی را با توجه به مجموعه اجراهای برنامه برای برش‌بندی ارائه داد[5]. این تکنیک جدید برش‌بندی، حاصل ترکیب موارد آزمون به همراه برش‌های برنامه است. این رویکرد برش پویای همزمان نامیده می‌شود زیرا از طریق مجموعه‌ای از موارد آزمون، تکنیک برش پویا را گسترش می‌دهد و به طور همزمان اعمال [27] و برش‌های قابل اجرایی تولید می‌کند که فقط بر روی یک ورودی صحیح هستند. برش‌های همزمان بر روی موارد آزمون، با اجتماع برش‌های پویا بر روی موارد آزمون مولفه محاسبه نمی‌شوند. در واقع اجتماع برش‌های پویا نامعلوم است. به این معنی که اجتماع برش‌ها پویا، صحت همزمانی را در همه‌ی ورودی‌ها حفظ نمی‌کند[5].

Number	Code
3	int frequentRenterPoints = 0;
4	Enumeration rentals = _rentals. elements ();
6	while (rentals. hasMoreElements ())
	{
7	Rental each = (Rental)rentals. nextElement ();
9	if (each.getDaysrented>1)
10	frequentRenterPoints = frequentRenterPoints + 2;
	else
11	frequentRenterPoints ++;
	}

جدول ۶:نمونه‌ای از برش پویای همزمان

الگوریتم تکرارشونده‌ای برای محاسبه‌ی برش‌های پویای همزمان ارائه شده است[5]. با آغاز از مجموعه‌ی اولیه‌ای از عبارات، به تدریج برش پویای همزمان ساخته می‌شود و در هر تکرار از این الگوریتم، برش پویای همزمان بزرگتر می‌شود. این رویکرد با جدا کردن زیر مجموعه‌ای از عبارات که رفتار خاصی دارند به درک برنامه کمک می‌کند. برش پویای همزمان می‌تواند به عنوان نسخه‌ی ویرایش شده از روش وایلد و همکارانش در نظر گرفته شود [11]. در روش وایلد و همکارانش مسئله‌ی مکان‌یابی عملکردهای درون کد، با شناسایی ارتباط موجود مابین شیوه‌هایی که کاربر و برنامه‌نویس، برنامه را می‌بینند در نظر می‌گیرند. برش پویای همزمان می‌تواند به عنوان اصلاحیه از روش‌ها برای مکان‌یابی توابع بر مبنای موارد آزمون در نظر گرفته شود[11] زیرا آن جریان داده‌های برنامه را در نظر می‌گیرد و سپس اجازه می‌دهد مجموعه عبارات انتخاب شده کاهش یابد. برش پویای همزمان از برنامه P با شاخص ⟨(I1,I2,…,Im),S,V)⟩، نمایش داده می‌شود به طوریکه حاصل این شاخص هر برنامه‌ی قابل اجرا و صحیح P0 است که از برنامه P با حذف صفر عبارت یا بیشتر بدست می‌آید. به طوریکه Im به ورودی‌ها، S به عبارات درون برنامه و V زیر مجموعه‌ای از متغیرها در برنامه P اشاره می‌کند. جدول ۶ نمونه‌ای از برش پویای همزمان را با شاخص زیر نشان می‌دهد. همچنین مقدار اولیه برای متغیرهای I1 و I2 به شرح زیر است:

⟨(I1,I2,14, frequentRenterPoints)⟩

I1= {rental.hasMoreElement=true, each.getDaysrented = 1}

I2= {rental.hasMoreElement=true, each.getDaysrented = 2}

۷-۲- برش آمورفوس

برش‌های ایستا، پویا، پویای همزمان و کواسی برش‌هایی با محدودیت نحوی هستند در حالیکه برش آمورفیس بر مبنای محدودیت‌های معنایی برنامه ساخته شده است[14,15,16]. تکنیک‌های برش‌بندی با محدودیت‌های نحوی، بر حذف عباراتی از برنامه بر اساس شاخص برش تمرکز دارند و از این رو محدودیت‌های نحوی جملات برنامه، بعد از عملیات برش‌بندی باقی می‌ماند. در حالیکه برش آمورفوس با استفاده از تکنیک‌های تبدیل برنامه ساخته می‌شود و معنای برنامه را با توجه به شاخص برش حفظ می‌کند. برش‌های تشکیل شده به اندازه‌ی سایر برش‌ها بزرگ نیستند. برش آمورفیس با توجه به معیار برش، شکل بسیار ساده‌شده از برنامه است. این برش به درک، آنالیز و استفاده‌ی مجدد برنامه کمک می‌کند.

برش آمورفیس با حذف عبارات غیر مربوط ساخته نمی‌شود بلکه با توجه به شاخص برش، تبدیلات نحوی ممکن را با حفظ معنا اجرا خواهد کرد در نتیجه خروجی آن هرگز بزرگتر از برنامه اصلی نخواهد بود.

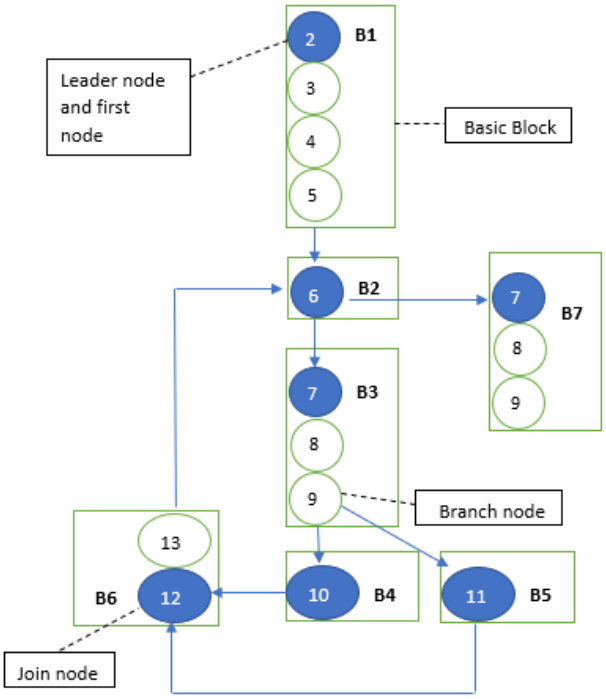
۸-۲- برش مبتنی بر بلوک

در الگوریتم‌های برش‌بندی سنتی از بدنه‌ی تابع به عنوان ناحیه‌ی برش‌بندی استفاده می‌شود به طوریکه عملیات برش‌بندی از عبارت مورد اشاره توسط شاخص برش آغاز شده و در بدنه‌ی تابع گسترش می‌یابد. با این وجود در زمینه‌ی استخراج توابع تک‌وظیفه‌ای، هدف آن است که برش حاصل به عنوان تابع جداگانه استفاده شود که این نوع از عملیات‌های برش‌بندی در سراسر بدنه‌ی تابع همیشه عملی نیست. از این‌رو اولین بار، برش مبتنی بر بلوک توسط ماریاما ارائه گردید[8]. برش مبتنی بر بلوک زیرمجموعه‌ای از برش ایستا است که با ساخت مناطق مبتنی بر بلوک، برش ایستا را در مرزهای یک بخش محدود می‌کند. از این رو با ساخت مناطق متفاوت امکان استخراج چند برش برای یک شاخص به‌وجود می‌آید و از میان آن‌ها برشی انتخاب می‌شود که اولاً حداقل عبارات را داشته باشد و دوماً رفتار برنامه را حفظ کند. در این بخش نحوه‌ی محاسبه‌ی برش مبتنی بر بلوک به طور مختصر ارائه می‌شود.

۲-۸-۱- محاسبه‌ی گراف جریان کنترلی

این گراف از تجزیه‌ی بدنه‌ی تابع به بلوک‌های پایه ایجاد می‌شود. هر بلوک پایه، دنباله‌ی متوالی عبارات برنامه است که جریان کنترلی از ابتدای بلوک وارد و از انتهای بلوک خارج می‌شود. توجه شود که امکان ایجاد شاخه یا توقف در میانه‌ی بلوک وجود ندارد. این گراف برای بهینه‌سازی کامپایلر و ابزارهای تحلیل ایستا مفید است.

الگوریتم بخش‌بندی مبتنی بر بلوک[12] اولین گره‌ی هر عبارت را به نام گره‌ی سردسته نام‌گذاری می‌کند. در این الگوریتم گره‌های پیوندی، گره‌هایی هستند که دو لبه یا بیشتر از دو لبه به آن‌ها وارد می‌شود همچنین گره‌های منشعب‌کننده گره‌هایی هستند که دو لبه یا بیشتر از دو لبه از آن خارج می‌شود. شکل شماره‌ی ۱ گراف جریان کنترلی تابع Statement از جدول ۱ را نشان می‌دهد.



شکل ۱:گراف کنترل جریان تابع Statement

۲-۸-۲- محاسبه‌ی گراف وابستگی کنترلی مبتنی بر بلوک

برای محاسبه گراف وابستگی کنترلی مبتنی بر بلوک مجموعه‌های تسلط(Dominant) و پس تسلط(postDominant) هر گره به ترتیب محاسبه می‌شوند و در انتها با توجه به مجموعه‌ی پس‌تسلط، گراف وابستگی کنترلی مبتنی بر بلوک محاسبه می‌شود.

برای محاسبه‌ی مجموعه‌های تسلط، گره‌های Start و Stop به ابتدا و انتهای گراف جریان کنترلی اضافه می‌شوند. سپس مجموعه‌ی تسلط یک گره چنین تعریف می‌شود:

گره‌ی N1 بر N2 مسلط است اگر هر مسیر از Start به N2 حتما از N1 عبور کند. به عنوان مثال مجموعه‌ی تسلط B4 از تابع Statement با توجه به شکل ۱ برابر است با:

Dominant(B4) = {B3, B2, B1}

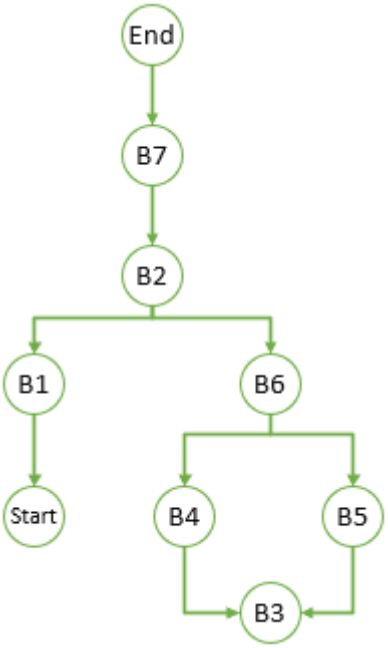
در گام بعد مجموعه‌ی PostDominant محاسبه می‌شود. مجموعه‌ی PostDomnant به صورت زیر تعریف می‌شود:

گره y بر گره z پس تسلط دارد (در گراف جریان کنترلی) اگر و تنها اگر هر مسیر موجود از گره z به گره stop از گره y عبور کند در این صورت گره y بر گره z پس‌تسلط دارد. به عنوان مثال مجموعه‌ی پس‌تسلط گره B2 به صورت زیر است:

postDominant(B2) = {B2, B3, B6}

پس از محاسبه‌ی مجموعه‌ی تسلط و پس‌تسلط نوبت به ساخت گراف وابستگی کنترلی مبتنی بر بلوک می‌رسد. برای ساخت این گراف، درخت پس‌تسلط ایجاد می‌شود که این درخت خود از مجموعه‌های پس تسلط ساخته می‌شود. شکل شماره‌ی ۲ گراف پس‌تسلط تابع Statement را نشان می‌دهد.

سپس در گراف جریان کنترل تمام لبه‌ها بررسی می‌شوند. برای هر لبه‌ی گراف جریان کنترل مانند AB، چنانچه گره‌ی A در مجموعه‌ی پس‌تسلط B نباشد این دو راس در درخت پس‌تسلط بررسی می‌شوند. بدین صورت که نزدیکترین جد مشترک در درخت پس تسلط بین رئوس A و B مورد جستجو قرار می‌گیرد و این جد مشترک C نامیده می‌شود. از گره‌ی B به سمت گره‌ی C حرکت کرده و تمام گره‌های مشاهده شده علامت‌دار می‌شوند. تمام گره‌های علامت‌دار، وابستگی کنترلی به A دارند.



شکل ۲:درخت پس‌تسلط تابع Statement

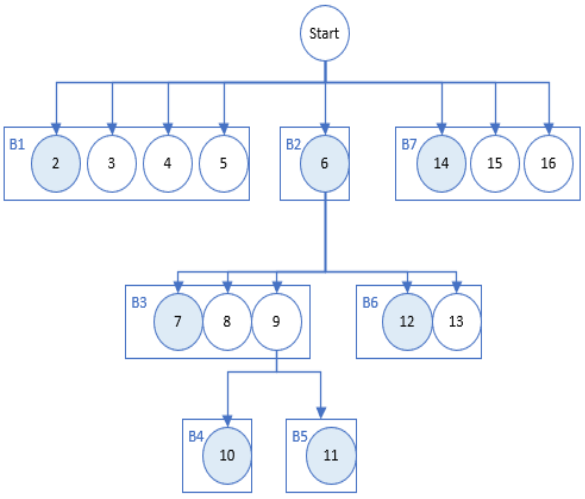
در نهایت پس از بررسی تمام لبه‌های گراف جریان کنترل و محاسبه‌ی تمام لبه‌های گراف وابستگی کنترل مبتنی بر بلوک، این گراف بدست می‌آید. شکل ۳ گراف وابستگی کنترلی را نشان می‌دهد.

۲-۸-۳- محاسبه‌ی بلوک‌های مرزی

برای محاسبه‌ی بلوک‌های مرزی، مجموعه‌های قابل‌دسترس، دام(Dom) و بلوک‌های مرزی محاسبه می‌شوند. بر طبق تعریف ماریاما، مجموعه‌ی قابل دسترس از روی گراف جریان کنترل محاسبه می‌شود. Reach(B) مجموعه‌ای از بلوک‌های اساسی است که از طریق بلوک B در جهت روبه‌جلو قابل دسترسی هستند بدون آنکه در گراف جریان کنترل، یال‌های بازگشتی در

نظر گرفته شوند [8]. به طور مثال مجموعه‌ی قابل دسترس برای B3 عبارت است از:

Reach(B3) = {B3, B4, B5, B6}



شکل ۳:گراف وابستگی کنترلی مبتنی بر بلوک تابع Statement

در گام بعد مجموعه‌ی دام(Dom) محاسبه می‌شود. این مجموعه از روی گراف وابستگی کنترل مبتنی بر بلوک محاسبه می‌شود. برای محاسبه‌ی مجموعه‌ی دام برای بلوکی مانند B_1، گره‌ی مسلط بر این بلوک در نظر گرفته می‌شود. فرض کنید این گره متعلق به بلوک B_2 است. بلوک‌هایی که به طور مستقیم یا غیرمستقیم از طریق گره‌ی سردسته‌ی بلوک B_2 در جهت روبه‌جلو قابل دسترس باشند مجموعه‌ی دام بلوک B_1 را تشکیل می‌دهند. به طور مثال مجموعه‌ی دام برای بلوک‌های زیر عبارتند از:

Dom(B1) = Dom(B2) = Dom(B3) = {B1, B2, B3, B4, B5, B6, B7}

در گام بعد مرزهای برش‌بندی برای یک عبارت مشخص و مجموعه‌های قابل دسترس و دام برای هر بلوک محاسبه می‌شود سپس عملیاتی که ذکر خواهد شد برای تمام بلوک‌های برنامه تکرار می‌شود این عملیات عبارت است از:

بلوکی با نام B_1 در نظر گرفته می‌شود. فرض می‌شود بلوک حاوی عبارت n بلوک B_2 باشد. چنانچه B_2 در هر دو مجموعه‌ی $Dom(B_1)$ و $Reach(B_1)$ وجود داشته باشد بلوک B_1 به مجموعه‌ی بلوک‌های مرزی عبارت n اضافه می‌شود. به طور مثال مجموعه بلوک‌های مرزی عبارت 8 از تابع Statement برابر است با:

Blocks (8) = {B1, B2, B3}

۴-۸-۲- محاسبه‌ی مناطق مبتنی بر بلوک

بر مبنای تعریف بلوک‌های قابل دسترس، ماریاما منطقه‌ی مبتنی بر بلوک را با $R(B_n)$ نشان می‌دهد که در آن B_n بلوک مرزی و شامل مجموعه‌ای از گره‌های متعلق به $Reach(B_n)$ است. با فرض آنکه شاخص برش (x,u) داده شده و x شماره‌ی خطی است که متغیر u در آن تعریف یا استفاده شده است برش در هر کدام از مناطق بلوک‌های مرزی عبارت x محاسبه می‌شود. بلوک‌های مرزی عبارت x با Blocks(x) نشان می‌دهند. از شاخص برش مشخص است نوع برش به برش ایستا اشاره دارد. شکل شماره‌ی ۴ مناطق مبتنی بر بلوک را با در نظر گرفتن شاخص (8, thisAmount) نشان می‌دهد. این مناطق R(B1)، R(B2) و R(B3) هستند. مناطق مبتنی بر بلوک بر اساس بلوک‌های مرزی معرفی شده در بخش قبل محاسبه می‌شوند.

Blocks (8) = {B_1, B_2, B_3}

```
1   public String statement ()
    {
2     double totalAmount = 0;
3     int frequentRenterPoints = 0;          R(B1)
4     Enumeration rentals = _rentals. elements ();
5     String result = "Rental Record\n";
6     while (rentals. hasMoreElements ())    R(B2)
      {
7       Rental each= (Rental)rentals.nextElement();
8       double thisAmount = each. get Charge ();
9        if ( each.getDaysrented>1)
            frequentRenterPoints =            R(B3)
10                  frequentRenterPoints + 2;
         else
11          frequentRenterPoints ++;
12      result = result + each. GetMovie().getTitle()
       + "\t" + String.valueOf(thisAmount) + "\n";
13       totalAmount = totalAmount + thisAmount;
       }
14      result += "Amount owed id: " +
                String.valueOf(totalAmount) + "\n";
15       result = "you earned "+
            String.valueOf(frequentRenterPoints) +
                        "frequent renter points";
16      return result;
      }
```

شکل ۴:مناطق مبتنی بر بلوک برای تابع Statement

۵-۸-۲- نقد برش‌ها و ارائه‌ی پیشنهادات

با توجه به آنکه مفاهیم مربوط به برش‌بندی و انواع برش روشن شد به سوالات زیر پاسخ داده می‌شود:

«چرا در استخراج توابع تک وظیفه‌ای از برش ایستا استفاده می‌شود؟ در حالی‌که معیارهای برش‌بندی، برش پویا را برای این کار پیشنهاد می‌دهند [8,21].

آیا امکان استفاده از برش‌های آمورفیس، پویای همزمان و کواسی برای استخراج توابع تک‌وظیفه‌ای وجود دارد یا خیر؟»

برش پویا شامل برخی از مسیرهای اجرایی یک متغیر خاص است. به عنوان مثال همانگونه که در بخش ۳-۲ مشاهده کردید عبارت شماره ۱۱ از برش‌بندی ایستا حذف شد. زیرا مسیر اجرا با توجه به مقدار متغیر each.getRenterPoints محدود شد و در استخراج توابع با عملکرد مجزا تمام مسیرهای اجرای متغیر پوشش داده می‌شود. برش ایستا تمام مسیرهای اجرای متغیر را پوشش می‌دهد. به این دلیل برخی رویکردهای پیشنهاد استخراج توابع مجزا [8, 21] از برش ایستا یا زیرمجموعه‌ای از برش ایستا(برش مبتنی بر بلوک) برای استخراج تابع تک‌وظیفه‌ای استفاده می‌شود. اما این سوال مطرح است که آیا برش ایستا کوچکترین برشی است که به عنوان تابع مجزا در نظر گرفته می‌شود؟ همچنین در استخراج توابع مجزا لازم است کوچکترین برش در نظر گرفته شود؟ در پاسخ به سوالات مطرح شده

معیارهای برش‌بندی، برشی را پیشنهاد می‌کنند که حداقل عبارات ممکن را داشته باشد. اما در استخراج توابع باید به دو نکته توجه کرد. نکته‌ی اول آن است که برش عملکرد خاصی را ارائه می‌دهد و بنابراین برش‌هایی مانند برش پویای همزمان و برش کواسی که هنگام برش‌بندی بیش از یک متغیر یا مسیرهای اجرایی خاصی از یک متغیر را در نظر می‌گیرند برای عملیات استخراج توابع مجزا مناسب نیستند. همچنین توابع حاصل از برش پویای همزمان و آمورفوس حداقل دو متغیر را به عنوان خروجی برمی‌گردانند در حالیکه هر تابع می‌تواند یک متغیر را به عنوان مقدار نهایی برگرداند. نکته دوم آن است که هر چه عبارات برش کمتر باشد زمان اجرای آن برش در مقایسه با سایر برش‌های مشابه از همان متغیر کمتر است بنابراین به شرط رعایت عملکرد مجزا، پوشش دادن تمام مسیرهای اجرایی یک متغیر و حفظ رفتار برنامه، برشی که کوچکتر است برای استخراج تابع مجزا بهتر است. برش مبتنی بر بلوک تقریبا چنین شرایطی را رعایت می‌کند. یعنی آنکه در ابتدا مناطق مشخص می‌شوند و عملیات برش‌بندی در این مناطق انجام و سپس برش‌بندی از داخلی‌ترین منطقه‌ی محاسبه می‌شود. چنانچه این برش بتواند رفتار برنامه را حفظ کند عملیات برش‌بندی به اتمام می‌رسد در غیر اینصورت عملیات برش‌بندی برای همان متغیر در منطقه‌ی بزرگتر ادامه می‌یابد.

در مجموع با توجه به مطالبی که گفته شد برای عملیات استخراج توابع تک وظیفه‌ای، برش ایستا در مقایسه با برش پویا مفیدتر است. همچنین برش‌های ایستای روبه‌عقب از برش‌های ایستای روبه‌جلو مناسب‌تر هستند.

۳- ارزیابی معیارهای استخرج متد

نیکولاس تسانتالیس و همکارانش [21] عقیده دارند عملیات بازسازی توابع چند وظیفه‌ای زمانی مثبت تلقی می‌شوند که ضمن حفظ رفتار برنامه، بر شاخص‌های کیفی نرم‌افزار تاثیرگذار باشند به طوریکه این تاثیرات مورد تایید توسعه‌دهندگان و طراحان نرم‌افزار باشند. با توجه به موارد گفته شده ایشان فرضیه‌ای را مطرح می‌کنند که عبارت است از:

« فرصت‌های شناسایی شده به منظور بازسازی تابع چند وظیفه‌ای، بر روی بخش‌هایی از کد تمرکز داشته که عملکرد مستقل و مجزایی دارند و کاربرد آن برطرف کردن نقض طراحی است [21] .»

یکی از نقص‌های طراحی، حسادت ویژگی (Feature envy) است. زمانیکه فیلدهای یک شی توسط شی دیگر فراخوانی شود و عملیاتی در شی ثانویه با فیلدهای شی اولیه انجام شود در اینجا حسادت ویژگی رخ می‌دهد. تجربه‌ی نیکولاس تسانتالیس و همکارانش[21] نشان می‌دهد که هنگام بازسازی امکان شکل‌گیری حسادت ویژگی وجود دارد.

همچنین ایشان معیارهایی برای انتخاب منبع کد جهت بررسی رویکردهای جدید بازسازی توابع مطرح نمودند که عبارتند از:

۱- منبع کد پروژه به طور عمومی در دسترس همه باشد.
۲- پروژه باید به اندازه‌ای بزرگ باشد تا فرصت‌های بازسازی به حد کافی به وجود بیاید.
۳- پروژه باید از پوشش آزمون بالایی برخوردار باشد تا بتوان پس از هر بازسازی ، برنامه را از جهت حفظ رفتار، مورد بررسی قرار داد.

۳-۱- بررسی معیار انسجام

مطالعه‌ی تجربی مایزر و بینکلی [17] نشان می‌دهد که معیارهای برش‌بندی موثر بودن یا نبودن عملیات بازسازی را بررسی می‌کنند. برای اینکار معیارهای مربوط به انسجام، قبل از عملیات بازسازی تابع، محاسبه می‌شوند تا این مقادیر با مقادیر محاسبه شده بعد از عملیات بازسازی تابع برای هر دو تابع باقیمانده و جدید مقایسه شوند. در این خصوص نیکولاس تسانتالیس و همکارانش [21] فرضیه‌ای را مطرح می‌کنند:

«استفاده از فرصت‌های شناسایی شده جهت بازسازی تابع چند وظیفه‌ای، انسجام کد آسیب دیده را بهبود می‌بخشد[21].»

به منظور اندازه‌گیری معیارهای انسجام مبتنی بر برش‌بندی از معیار معرفی شده توسط لیندا و جفری [24] استفاده می‌شود.

فرض می‌شود V_M مجموعه‌ای از متغیرهای استفاده شده توسط ماژول M و V_O زیر مجموعه‌ای از V_M است که شامل متغیرهای خروجی M است. همچنین a متغیری بازگردانده شده توسط M ، b متغیر سراسری اصلاح شده توسط M و c پارامتری است که از طریق ارجاع به تابع پاس داده و به وسیله‌ی M اصلاح می‌شود. سرانجام SL_i برش محاسبه شده برای متغیر $v_i \in V_O$ است و SL_{int} اشتراک SL_i در سراسر ماژول M برای $v_i \in V_O$ است.

$$Overlap(M) = \frac{1}{|V_O|}\sum_{i=1}^{|V_O|}\frac{|SL_{int}|}{|SL_i|}$$

$$Tightness = |SL_int|/(Length(M))$$

$$Coverage(M) = \frac{1}{|V_O|}\sum_{i=1}^{|V_O|}\frac{|SL_i|}{lenght(M)}$$

فشردگی (*Tightness*) بیانگر نسبت تعداد عبارات مشترک سراسر برش‌ها در طول ماژول است، در حالی که هم‌پوشانی (Overlap) نسبت متوسط تعداد عبارات مشترک در همه برش‌ها به اندازه هر برش را نشان می‌دهد. هر چه فشردگی و هم‌پوشانی بالاتر باشد انسجام کد بیشتر است. در ماژول با فشردگی یا هم‌پوشانی بالا، تعداد عبارات یکسان بین تابع استخراجی و باقی‌مانده بعد از عمل استخراج برش، بزرگ خواهد بود. از طرف دیگر، هم‌پوشانی(Coverage) میانگین اندازه‌ی برش را در سراسر طول ماژول بیان می‌کند و بنابراین به طور غیر مستقیم با درجه‌ی جملات مشترک در میان برش‌ها ارتباط دارد. با این وجود زمانی همپوشانی زیاد است که برش‌های استخراجی در بخشی از ماژول گسترش یابند. این معیار به طور غیر مستقیم وجود چندین جمله‌ی مشترک در میان برش‌ها را نشان می‌دهد.

۳-۲- معیار Precision و Recall

در این بخش توابع استخراج شده توسط متخصص با توابع استخراج‌شده به وسیله‌ی رویکردهای استخراج تابع تک‌وظیفه‌ای مورد مقایسه قرار می‌گیرند. تسانتالیس و همکارانش [21] شاخص‌های TP، FP و FN را به صورت زیر تعریف می‌کنند:

TP (True Positive): نمایش‌دهنده‌ی تعداد فرصت‌های بازسازی توابع چند وظیفه‌ای است که به طور مشترک توسط متخصص و الگوریتم برش‌بندی شناسایی شده است.

FP (False Positive): نمایش‌دهنده‌ی تعداد فرصت‌های بازسازی توابع چند وظیفه‌ای است که به وسیله‌ی الگوریتم برش‌بندی شناسایی می‌شود اما متخصص آن را شناسایی نکرده است.

FN (False Negative): نمایش‌دهنده‌ی فرصت‌های بازسازی توابع چند وظیفه‌ای است که به وسیله‌ی متخصص شناسایی می‌شود اما الگوریتم برش‌بندی آن را شناسایی نکرده است.

در توابع چندوظیفه‌ای توسعه‌دهندگان و متخصصان طراحی نرم‌افزار نمی‌توانند همه‌ی وابستگی‌های داده‌ای و کنترلی بین جملات و متغیرهای برنامه را بررسی کنند. اما رویکردهای استخراج توابع تک وظیفه‌ای مبتنی بر برش به خوبی این امر را تحلیل می‌کنند. تجربه‌ی تسانتالیس و همکارانش [21] نشان می‌دهد که متخصصان می‌توانند خطوط کدی که در کنار هم هستند را به عنوان یک تابع تک وظیفه‌ای در نظر بگیرند. این در حالی است که یک تابع چند وظیفه‌ای که بیش از ۱۰۰ خط کد داشته باشد امکان استخراج تابع تک وظیفه‌ای را به وسیله‌ی نیروی انسانی محدود می‌کند. بنابراین ایجاد خطای انسانی توسط متخصص یا توسعه‌دهنده‌ی نرم‌افزار در هنگام استخراج توابع تک وظیفه‌ای وجود دارد.

۴- بحث و نتیجه‌گیری

در این مقاله موضوع استخراج توابع تک‌وظیفه‌ای مورد بررسی قرار گرفت. برای استخراج توابع تک‌وظیفه‌ای رویکردهای متفاوتی وجود دارد. یکی از پرطرفدارترین رویکردهای موجود، استخراج توابع مجزا با استفاده از برش‌بندی کد است. زیرا برش‌بندی با در نظر گرفتن وابستگی‌های داده‌ای و کنترلی می‌تواند به صورت نحوی یا معنایی، توابعی را با عملکرد مجزا استخراج کند که به نوبه‌ی خود یکی از مناسب‌ترین رویکردهای استخراج تابع است.

برش ایستا، پویا یا ترکیبی از هر دو ویژگی است. برش ایستا عباراتی از برنامه را شامل می‌شود که بر مقدار متغیر تاثیرگذار است در حالیکه برش پویا با در نظر گرفتن ورودی‌های مشخص برای متغیر، عباراتی از برنامه را در نظر می‌گیرد که بر متغیر تاثیرگذار است. همچنین برش، روبه‌جلو یا روبه‌عقب است. در برش روبه‌عقب عباراتی محاسبه می‌شوند که شاخص برش را تحت تاثیر قرار می‌دهند در حالیکه در برش روبه‌جلو عبارتی محاسبه می‌شوند که تحت تاثیر شاخص برش قرار می‌گیرند. همچنین برش قابل اجرا یا غیر قابل اجرا است.

برش‌ها انواع مختلفی دارند. پرکاربردترین انواع برش‌ها در این مقاله مورد بررسی قرار گرفته است. برش‌های مورد بررسی عبارتند از: برش ایستا، برش پویا، برش کواسی، برش پویای همزمان، برش آمورفوس، برش مبتنی بر بلوک.

در این مقاله موضوع امکان‌سنجی جایگزینی سایر برش‌ها به‌جای برش ایستا مطرح و اثبات شد که برش ایستا و برش مبتنی بر بلوک با توجه به اینکه همه‌ی مسیرهای اجرا را در نظر می‌گیرند نسبت به برش پویا مناسب‌تر هستند. همچنین مسئله‌ی جایگزینی برش ایستای روبه‌جلو به جای برش ایستای روبه‌عقب مطرح و مشخص شد که برش روبه‌عقب از برش روبه‌جلو مناسب‌تر و پرکاربردتر است.

در این مقاله معیارهای کیفی و کمی اندازه‌گیری برش معرفی شدند. بر طبق فرضیه‌ی نیکولاس تسانتالیس و همکارانش[21] فرصت‌های شناسایی شده به منظور بازسازی توابع چند وظیفه‌ای، بر روی بخش‌هایی از کد تمرکز داشته که عملکرد مستقل و مجزایی دارند و کاربرد آن برطرف کردن نقض طراحی است. همچنین معیار انسجام به عنوان یک معیار کمی معرفی شد و مفاهیم مربوط به این معیار از جمله همپوشانی، فشردگی و پوشش به طور کامل تشریح شد. سپس معیارهای Precision و Recall مورد بررسی قرار گرفتند.

برشی که به عنوان تابع تک‌وظیفه‌ای در نظر گرفته می‌شود باید مورد تایید توسعه‌دهندگان و طراحان نرم‌افزار قرار گیرد. از طرف دیگر اگر کار استخراج تابع تک‌وظیفه‌ای توسط توسعه‌دهندگان نرم‌افزار انجام شود ممکن است توسعه‌دهندگان در مواجه با توابع چند وظیفه‌ای با تعداد خطوط بالا نتوانند وابستگی‌های داده‌ای و کنترلی را به خوبی تحلیل کنند و از این رو در استخراج توابع با عملکرد مجزا دچار خطا می‌شوند. پیشنهاد نویسندگان مقاله ترکیبی از دو روش است. الگوریتم‌های برش‌بندی فرصت‌هایی را به عنوان توابع تک‌وظیفه‌ای شناسایی می‌کنند و به توسعه‌دهندگان پیشنهاد می‌دهند. چنانچه این توابع مورد تایید توسعه‌دهندگان باشند به عنوان توابع تک وظیفه‌ای مورد استفاده قرار می‌گیرند.

سپاسگزاری

به مصداق «من لم یشکر المخلوق لم یشکر الخالق» بسی شایسته است از استاد فرهیخته و فرزانه جناب آقای دکتر سعید پارسا که با کرامتی چون خورشید، سرزمین دل را روشنی بخشیدند و گلشن سرای علم و دانش را با راهنمایی‌های کار ساز و سازنده بارور ساختند؛ تقدیر و تشکر نماییم.

۵- منابع

[1] Weiser, "Program Slicing," *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING*,, pp. 352-357, JULY 1984.

[2] M. Harman, "A piece of cake".

[3] M. Kamkar, "An Overview and Comparative Classification of," *Journal of Systems and Software*, vol. 31, no. 3, pp. 197-214, 1995.

[4] G. Venkatesh, "The semantic approach to program slicing," *ACM SIGPLAN Notices*, vol. 26, pp. 107-119, 1991.

[5] R. Hall, "Automatic extraction of executable program subsets by simultaneous program," *Journal of Automated Software Engineering*, vol. 2, pp. 33-53, 1995.

[6] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship, Prentice Hall , PTRUpper Saddle River, NJ ,United States, 2008, p. 448.

[7] D. Binkley, "The application of program slicing to regression testing," *Information and Software Technology*, vol. 40, no. 11-12, pp. 583-594, 1998.

[22] T. Reps and T. Bricker, "Illustrating interference in interfering versions of programs," *Proceedings of the 2nd International Workshop on Software configuration management,* p. 46–55, October 1989.

[23] K. Gallagher and J. Lyle, "Using program slicing in software maintenance," *IEEE Transactions on Software Engineering,* pp. 751-761, 1991.

[24] L. M. Ott and J. J. Thuss, "The relationship between slices and module cohesion," *Proceedings of the 11th international conference on Software engineering,* p. 198–204, May 1989.

[25] L. Xu, B. Xu, Z. Chen, J. Jiang, H. Chen and a. H. Yang, "Regression testing for web applications based on slicing," *In Proceedings of 28th IEEE Annual International Computer Software and Applications Conference,* pp. 652-656, 2003.

[26] S. B. Horwitz, T. Reps and D. Binkley, "Interprocedural slicing using dependence graphs," *ACM Transactions on Programming Languages and Systems,* pp. 26-61, 1990.

[27] N.Sasirekha, A. Robert and M.Hemalatha, "PROGRAM SLICING TECHNIQUES AND ITS," *International Journal of Software Engineering & Applications (IJSEA),* vol. 2, November 2011.

[28] S. Horwitz, T. Reps and D. Binkley, "Interprocedural slicing using dependence graphs," *Proceedings of the ACM SIGPLAN 1988 conference on Programming language design and implementation,* pp. 35-46, June 1988.

[29] J. Ferrante, K. J. Ottenstein and J. D. Warren, "The program dependence graph and its use in optimization," *ACM Transactions on Programming Languages and Systems,* vol. 9, pp. 319-349, Novamber 1987.

[8] K. Maruyama, "Automated method-extraction refactoring by using blockbased slicing," *Proceedings of the Symposium on Software Reusability,* pp. 31-40, 2001.

[9] D. Binkley, S. Danicic, T. G. thy, M. Harman, A. k. Kiss and B. Korel, "Minimal Slicing and the Relationships Between Forms of Slicing," *Proceedings of the Fifth IEEE International Workshop on Source Code Analysis and Manipulation,* 2005.

[10] B. Korel and J. Laski, "Dynamic slicing of computer programs," *The Journal of Systems,* vol. 2, pp. 187-195, 1990.

[11] N. Wilde, J. Gomez, T. Gust and a. D. Strasburg, "Locating user functionality in old code," *in Procs of Intern. Conf. on Soft. Maint., Orlando, Florida,U.S.A.,IEEE CS Press,* pp. 200-205, 1992.

[12] A. Aho, R. Sethi, J. Ullman and A. wesley, "Compilers: Principles, Techniques, and Tools," *Addison-Wesley,* 1986.

[13] Z. Chen, Y. Zhou, B. Xu, J. Zhao and H. Yang, "A novel approach for measuring class cohesion based on dependence analysis," *In Proceedings of International Conference on Software Maintenance,* pp. 377-384, October 2002.

[14] M. Harman and S. Danicic, "Amorphous program slicing," *IEEE International Workshop on Program Comprehension, IEEE Computer Society,Washington,DC,* pp. 70-79, May 1997.

[15] M. Harman and S. Danicic, "Using program slicing to simplify testing," *Journal of Software Testing, Verification and Reliability,* vol. 5, no. 3, pp. 143-162, 1995.

[16] D. Binkley, L. R. Raszewski, C. Smith and M. Harman, "An empirical study of amorphous slicing as a program comprehension support tool," *8th IEEE International Workshop on Program Comprehension (IWPC 2000),Limerick, Ireland.,* pp. 161-170, 2000.

[17] T. M. Meyers and D. Binkley, "An empirical study of slice-based cohesion and coupling metrics," *ACM Transactions on Software Engineering and Methodology,* December 2007.

[18] M. M. A. Abdallah and H. A. Tamimi, "Clauser: Clause Slicing Tool for C Programs," *International Journal of Software Engineering and Its Applications,* vol. 10, pp. 49-56, Novamber 2016.

[19] B. KOREL and J. LASKI, "Daynamic program slicing," *Information procrssing letters,* pp. 155-163, 11 Septtember 1987.

[20] D. B. Copeland, Hate, SOLID is not Solid: Five Object-Oriented Principles To Create a Codebase Everyone Will, David Bryant Copeland, 2020, p. 70.

[21] N. Tsantalis and A. Chatzigeorgiou, "Identification of extract method refactoring opportunities for the," *The Journal of Systems and Software,* vol. 84, no. 10, pp. 1757-1782, 14 May 2011.